SKYNORA

TECHNICAL SPECIFICATION & ENGINEERING BLUEPRINT

Product Requirements Document (PRD)

Enterprise Client-Side Flight Booking Engine, Customer Loyalty Portal & Administrative Control Deck.

AUTHOR & ROLE
Senior Frontend Architect & Tech Lead

DOCUMENT VERSION
v1.1.0 (Skynora Production Blueprint)

TECHNOLOGY ARCHITECTURE
Vanilla HTML5 · Modern CSS3 · ES6+ Modules

BRAND IDENTITY
Skynora (Formerly SkyVoyage)

TARGET RELEASE
Q4 2026 Core Delivery

CURRENT COMPLETION STATE
~45% Overall (Phase 2 Active)

Current Sprint Focus: Phase 2 Core Logic Implementation

Visual design tokens, UI layouts, and mock datasets are 100% complete under the Skynora brand. Engineering focus is actively directed toward client-side JavaScript controllers, OOP domain models, and state persistence.

1. Executive Summary & Vision

1.1 Project Overview

Skynora is an enterprise-class, responsive, client-side flight booking web platform engineered to deliver an uncompromising, seamless travel discovery, reservation, and itinerary management experience. Built strictly with modern native web standards (**Semantic HTML5, CSS3 Custom Properties/Tokens, and ES6+ Native Modules**), the system operates entirely without external heavy frontend frameworks (React, Angular, Vue) or bundling steps.

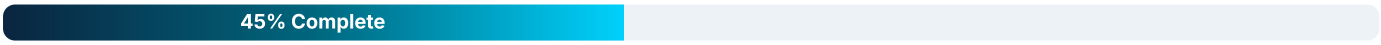
1.2 Core Architectural Tenets

- **Zero-Dependency Performance:** 100% native browser APIs ensure near-instant first paint times, sub-second route transitions, and low memory footprints.
- **Separation of Concerns:** Explicit isolation between presentation (CSS), markup (HTML), domain business rules (OOP Classes), and data transport (Data Service).
- **Modular ES6 Architecture:** Component controllers dynamically interact using standard ES6 `import` and `export` statements.
- **Stateful Client-Side Persistence:** Full booking state machine, user accounts, and session contexts are persistently maintained across browser lifecycles via `LocalStorage` and `SessionStorage`.

2. Current Project Status & Progress Tracker

2.1 Implementation Health: ~45% Completed

The project has completed its visual layout and design tokenization phase. The underlying markup and stylesheets for major screens are ready and verified. JavaScript application controllers and domain models are ready for immediate implementation.



Layer / Architectural Domain	Progress	Status	Detailed Scope State
1. Design System & Tokens	100%	COMPLETED	Color palette, typography tokens, elevation, and layout variables in `variables.css`.
2. Mock Data Repositories	100%	COMPLETED	Comprehensive datasets for 15+ airports, 10 global airlines, and 20+ flight schedules.
3. HTML View Templates	85%	IN PROGRESS	9 views created; dedicated `search.html` and `seat-selection.html` to be authored.
4. CSS Stylesheets & Responsive Rules	90%	COMPLETED	7 dedicated page stylesheets completed with mobile-first breakpoints.
5. OOP Domain Entity Models	0%	PENDING	`Flight.js`, `Seat.js`, `Booking.js`, `User.js` skeletons defined.
6. Data Access Layer & Services	0%	PENDING	Unified `data-service.js` with Fetch API and local storage caching.
7. Page Controllers & Interactivity	0%	PENDING	Search filter logic, dynamic seat matrix, form validation, and checkout.
8. State Persistence Layer	0%	PENDING	Syncing bookings and active user sessions to `LocalStorage`.

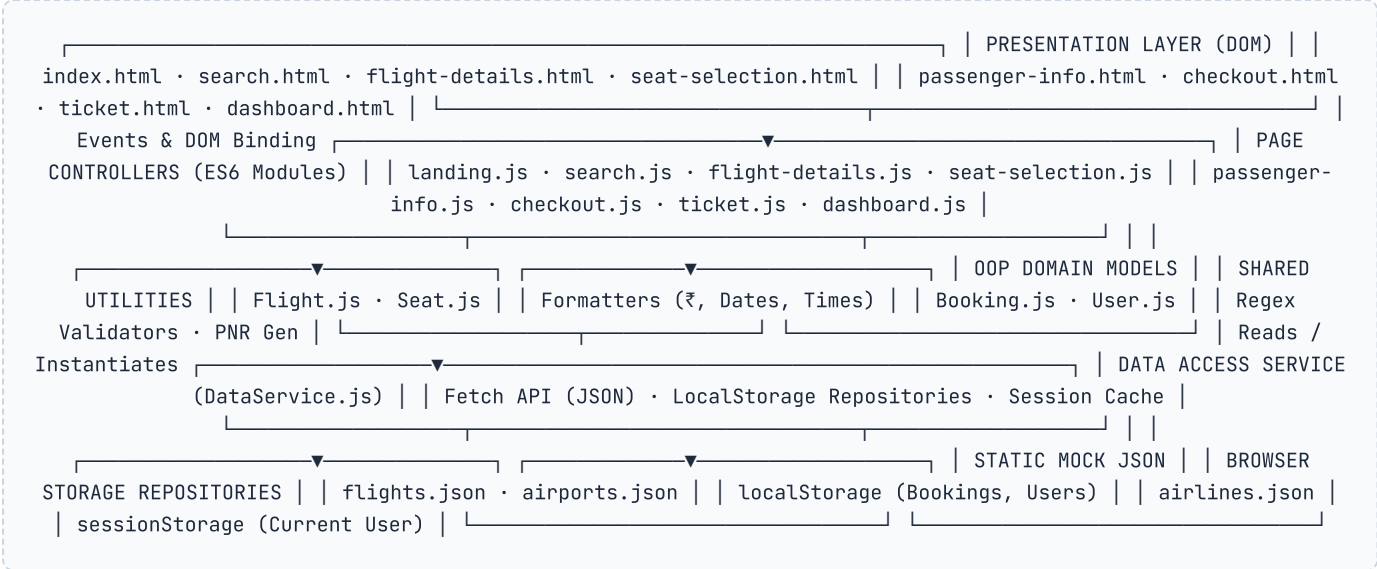
2.2 Detailed Module Audit & Action Matrix

Page / Component	File Location	Markup & CSS	JS Controller	Immediate Engineering Action
Landing Page	`index.html`	100%	0%	Bind trip type tabs, origin/destination autocomplete dropdowns, submit routing.
Search Results	`search.html` (Missing)	CSS ONLY	0%	Create `search.html` view, connect `search.css`, write faceted filter engine.
Flight Details	`flight-details.html`	100%	0%	Parse `?id=FLxxx` URL parameter, render route timeline and amenities.
Cabin Seat Selection	`seat-selection.html` (Missing)	CSS ONLY	0%	Create `seat-selection.html` view, render 3-3 seat grid, handle seat surcharge.
Passenger Information	`passenger-info.html`	100%	0%	Dynamic traveler sub-forms, email and phone regex validation, package draft order.
Checkout & Payment	`checkout.html`	100%	0%	Tab selector (Card/UPI/NetBanking), Luhn card algorithm, order commitment.
Confirmation & E-Ticket	`ticket.html`	100%	0%	Render official boarding pass, PNR barcode generation, window.print() formatting.
User Authentication	`login.html`, `register.html`	100%	0%	Validate credentials against `LocalStorage`, manage session state.
Customer Dashboard	`dashboard.html`	100%	0%	Query user bookings, upcoming vs past filters, cancellation state change.
Admin Control Deck	`admin.html`	100%	0%	KPI computation (Revenue, Bookings), flight schedule CRUD, user lookup.

> **Senior Engineer Note on Immediate Gaps:** Notice that while `css/pages/search.css` and `css/pages/seat-selection.css` are fully styled, their corresponding HTML views (`search.html` and `seat-selection.html`) must be generated in Sprint 1 to ensure the booking funnel links operate without HTTP 404 errors.

3. Target Frontend Architecture Blueprint

3.1 Layered Architectural Model



3.2 Full Repository Directory Specification

```
Skynora-Frontend-Team-Project/
├── PRD.pdf                          # Official Compiled PDF Specification
├── README.md                        # Repository overview & setup guide
│
├── ── HTML Views ──────────────────
├── index.html                       # Landing page with flight search engine
├── search.html                      # [Sprint 1] Search results & faceted filters
├── flight-details.html              # Detailed itinerary & cabin choices
├── seat-selection.html              # [Sprint 1] Interactive cabin seat matrix
├── passenger-info.html              # Traveler & contact information forms
├── checkout.html                    # Payment portal (Card, UPI, NetBanking, Wallet)
├── ticket.html                      # Confirmed E-Ticket & printable boarding pass
├── login.html / register.html       # Authentication portals
├── dashboard.html                   # User trips feed, status tabs & Skynora Points
├── admin.html                       # Admin operations, flight fleet & user directory
│
├── ── CSS Design System ───────────
├── css/
│   ├── reset.css                    # Box-sizing & browser normalize resets
│   ├── variables.css                # Design tokens (Colors, Typography, Elevation)
│   ├── main.css                     # Shared components (Navbar, Footer, Buttons, Grid)
│   └── pages/                       # Scoped stylesheets
│       ├── landing.css              # Hero & promo showcases
│       ├── search.css               # Filter sidebar & flight cards
│       ├── flight-details.css       # Route timeline & amenities
│       ├── seat-selection.css       # Interactive seat grid & legend
│       ├── booking.css              # Stepper header, forms & ticket styling
│       ├── auth-dashboard.css       # Auth split screen & dashboard cards
│       └── admin.css                # Admin tables, KPIs & sidebar
│
├── ── Modular JavaScript ──────────
├── js/
│   ├── app.js                       # Application bootstrapper & global navbar
│   ├── data-service.js              # Unified Data Access Layer (Fetch + LocalStorage)
│   ├── utils.js                     # Currency formatting, PNR generator, date tools
│   │
│   ├── classes/                     # OOP Domain Models
│   │   ├── Flight.js                # Flight entity & pricing logic
│   │   ├── Seat.js                  # Seat entity & status tracking
│   │   ├── Booking.js               # Booking order entity & PNR
│   │   └── User.js                  # User profile & loyalty points
│   │
│   └── pages/                       # View Controllers
│       ├── landing.js               # Search widget & airport autocomplete
│       ├── search.js                # Dynamic filter & flight card render
│       ├── flight-details.js        # Flight details population
│       ├── seat-selection.js        # Seat matrix generator & pricing sync
│       ├── passenger-info.js        # Passenger form validation
│       ├── checkout.js              # Payment processing simulation
│       ├── ticket.js                # E-Ticket population & print handler
│       ├── auth.js                  # Sign in & registration handler
│       ├── dashboard.js             # User trips feed & status filters
│       └── admin.js                 # Flight manager CRUD & KPI metrics
│
└── ── Mock Datasets ───────────────
    ├── airlines.json                # Airline codes, names, ratings
    ├── airports.json                # Airport IATA codes & city locations
    └── flights.json                 # Master flight schedules & pricing
```

4. End-to-End User Journeys

4.1 Primary Flight Booking Funnel

```
[1. index.html] ————————> [2. search.html] ————————> [3. flight-details.html] Search Origin, Dest, Filter by Price, Stops, Inspect Schedule, Fare Class, Dates & Passengers Airlines, Duration Baggage & In-flight Amenities | ▼ [4. seat-selection.html] -> [5. passenger-info.html] -> [6. checkout.html] -> [7. ticket.html] Interactive Aircraft Traveler Names, DOB, Credit Card, UPI, Printable Boarding Cabin & Surcharge Contact Email & Phone Order Summary Sync Pass & PNR Code
```

4.2 User Authentication & Profile Lifecycle

```
Visitor —> [register.html] —> Save Account in LocalStorage | ▼ Visitor —> [login.html] ——> Set Active Session in SessionStorage | ▼ [dashboard.html] - View Upcoming & Past Trips - Inspect Skynora Points Balance - Cancel Reservation (Updates Status)
```

5. Domain Modeling & Technical Data Contracts

5.1 Domain Classes (OOP Entities)

Class: Flight (`js/classes/Flight.js`)

Represents an airline schedule record. Encapsulates dynamic fare calculation per cabin class.

```
export class Flight {
  constructor({ id, airline, flightNumber, origin, destination,
                departureTime, arrivalTime, duration, price, stops,
                aircraft, seatsAvailable, amenities }) {
    this.id = id;
    this.airline = airline;
    this.flightNumber = flightNumber;
    this.origin = origin;
    this.destination = destination;
    this.departureTime = departureTime;
    this.arrivalTime = arrivalTime;
    this.duration = duration;
    this.price = price; // { economy: number, business: number, firstClass: number }
    this.stops = stops;
    this.aircraft = aircraft;
    this.seatsAvailable = seatsAvailable;
    this.amenities = amenities || [];
  }

  getPriceForClass(cabinClass = 'economy') {
    return this.price[cabinClass] || this.price.economy;
  }
}
```

Class: Seat (`js/classes/Seat.js`)

Represents an individual aircraft seat on a specific row and column with surcharge attributes.

```
export class Seat {
  constructor({ id, row, col, cabinClass, price, isOccupied = false, isSelected = false }) {
    this.id = id;             // e.g. "12A"
    this.row = row;           // 12
    this.col = col;           // "A"
    this.cabinClass = cabinClass; // "business" | "economy"
    this.price = price;       // Surcharge (₹0 for standard, ₹600 for extra legroom, etc.)
    this.isOccupied = isOccupied;
    this.isSelected = isSelected;
  }

  toggleSelect() {
    if (this.isOccupied) return false;
    this.isSelected = !this.isSelected;
    return true;
  }
}
```

Class: Booking (`js/classes/Booking.js`)

Encapsulates an accepted travel reservation with passenger rosters, payment metadata, and PNR.

```
export class Booking {
  constructor({ id, pnr, flightId, flightDetails, passengers,
                contact, seats, cabinClass, pricing, paymentMethod,
                status = 'confirmed', createdAt = new Date().toISOString() }) {
    this.id = id || 'BK-' + Date.now();
    this.pnr = pnr;
    this.flightId = flightId;
    this.flightDetails = flightDetails;
    this.passengers = passengers; // [{ name, age, gender, seatNumber }]
    this.contact = contact;       // { email, phone }
    this.seats = seats;           // ["12A", "12B"]
    this.cabinClass = cabinClass;
    this.pricing = pricing;       // { baseFare, seatFees, tax, total }
    this.paymentMethod = paymentMethod;
    this.status = status;         // 'confirmed' | 'completed' | 'cancelled'
    this.createdAt = createdAt;
  }
}
```

5.2 Browser Storage Architecture

Storage Type	Storage Key	Data Schema / Object Type	Purpose
`localStorage`	`skynora_bookings`	`Array<Booking>`	Permanent booking history across sessions (fallback: `skyvoyage_bookings`).
`localStorage`	`skynora_users`	`Array<User>`	Registered accounts with hashed passwords and Skynora loyalty points.
`localStorage`	`skynora_custom_flights`	`Array<Flight>`	Admin-created flights appended to base schedules.
`sessionStorage`	`skynora_current_user`	`User Object`	Active authenticated user session.
`sessionStorage`	`skynora_active_booking`	`Draft Booking State`	Temporary booking funnel draft across stepper pages.

6. Implementation Roadmap & Sprint Schedule

Sprint	Target Milestone	Key Deliverables	Timeline
Sprint 1	Core Data Layer & Missing Views	Create `search.html` & `seat-selection.html`; code domain classes; build `data-service.js` & `utils.js`.	Week 1
Sprint 2	Search Engine & Booking Funnel	Landing search autocomplete; search results filters (price slider, stops, airline); interactive seat map.	Week 2
Sprint 3	Checkout & E-Ticket Generation	Passenger validation; checkout payment simulation; PNR generation; printable e-ticket rendering.	Week 3
Sprint 4	User Dashboard & Admin Deck	User login/signup with session state; user trip history and Skynora points; admin flight CRUD and revenue KPIs.	Week 4
Sprint 5	QA, Polish & Accessibility	Keyboard navigation, ARIA labels, empty state fallbacks, cross-browser responsive audits.	Week 5

7. Frontend Engineering Standards & Quality Assurance

- **ES6+ Code Purity:** Strict usage of `const` / `let`, arrow functions, destructuring, and `async/await`. Zero inline HTML JS event attributes (no `onclick="..."`).
- **CSS Design Tokens:** All styling rules must consume variables from `variables.css` for brand colors, spacing, and typography to maintain visual consistency.
- **Accessibility (WCAG 2.1 AA):** All interactive components (tabs, buttons, form inputs) must have appropriate `aria-label`, `role`, and keyboard focus states.
- **Resilient Error Handling:** Loading states (`.loading-overlay`, `.spinner`) must display while data fetches resolve. Clean error messages for invalid search queries or failed validations.

Skynora Frontend Team Project Specification

Approved by Senior Frontend Architecture Lead